

Titre professionnel RNCP niveau 5 – Développeur web et web mobile

Dossier de projet

Urban PREDICT

Urban PREDICT

AccueilOutilVisualisationÀ proposAide

Outil

Données



1. Sélectionnez une commune.

Entrez un Code INSEE : Voir les données

Modèle de Machine Learning



2. Utilisez le modèle de Machine Learning.

Entraînez le modèle : Entraîner

Testez le modèle : Tester

Tony NEMOUTHE
07/03/2025

Sommaire

Liste des compétences mises en œuvres.....	4
Compétences professionnelles.....	4
Compétences transversales.....	4
Introduction.....	5
Contexte du projet.....	6
Expression des besoins du projet.....	6
Contraintes du projet.....	6
Les livrables attendus.....	6
Gestion de projet.....	7
Planning et organisation.....	7
Environnement humain.....	8
Environnement technique.....	8
Objectifs de qualité.....	9
Réalisations côté front-end.....	10
Création de la maquette de l'interface utilisateur.....	10
Schéma d'enchaînement des maquettes.....	10
Arborescence de l'application web.....	12
Charte graphique.....	12
Capture d'écran de l'interface utilisateur.....	14
Extraits de code d'interfaces utilisateur.....	15
Réalisations côté back-end.....	18
Base de données.....	18
Schémas de base de données.....	18
Script de création de la base de données.....	20
Extrait de code de composant métier.....	20
Extrait de code de composant d'accès aux données.....	22
Éléments de sécurité de l'application.....	23
Jeu d'essai.....	24
Tests côté front-end.....	24
Tests côté back-end.....	26
Veille sur les vulnérabilités de sécurité.....	29
Protections contre les attaques Cross-Site Scripting (XSS).....	29
Protections contre les injections SQL (SQLi).....	29
Protections contre les attaques Cross-Site Request Forgery (CSRF).....	29
Vulnérabilités de bibliothèques tierces.....	29
Documenter le déploiement.....	30
Solution d'hébergement.....	30
Script de déploiement.....	30
Étapes avant mise en production.....	31
Conclusion.....	33
Annexes.....	34

Index des figures

Figure 1 : Suivi des tâches via Trello.....	7
Figure 2 : Architecture et technologies.....	9
Figure 3 : Enchaînement des maquettes.....	11
Figure 4 : Arborescence de l'application web.....	12
Figure 5 : Charte graphique.....	13
Figure 6 : Extrait de code HTML de l'interface utilisateur.....	15
Figure 7 : Extrait de code CSS de l'interface utilisateur.....	16
Figure 8 : Extrait de code JavaScript de l'interface utilisateur.....	17
Figure 9 : Schéma conceptuel de la base de données.....	19
Figure 10 : Schéma physique de la base de données.....	19
Figure 11 : Script SQL pour la création de la base de données.....	20
Figure 12 : Extrait de code Python de composant métier.....	21
Figure 13 : Extrait de code Python de composant d'accès à la base de données.....	22
Figure 14 : Tests unitaires.....	24
Figure 15 : Liste des dépendances dans requirements.txt.....	30
Figure 16 : Étapes avant mise en production.....	31

Liste des compétences mises en œuvres

Compétences professionnelles

1. Développer la partie front-end d'une application web ou web mobile sécurisée :
 1. Installer et configurer son environnement de travail en fonction du projet web ou web mobile.
 2. Maquetter des interfaces utilisateur web ou web mobile.
 3. Réaliser des interfaces utilisateur statiques web ou web mobile.
 4. Développer la partie dynamique des interfaces utilisateur web ou web mobile.
2. Développer la partie back-end d'une application web ou web mobile sécurisée :
 5. Mettre en place une base de données relationnelle.
 6. Développer des composants d'accès aux données SQL et NoSQL.
 7. Développer des composants métier coté serveur.
 8. Documenter le déploiement d'une application dynamique web ou web mobile.

Compétences transversales

- Communiquer en français et en anglais.
- Mettre en œuvre une démarche de résolution de problème.
- Apprendre en continu.

Introduction

Durant ma formation chez Holberton School, j'ai eu l'opportunité de réaliser le développement d'une application web intitulée **Urban PREDICT**. Cette application web permet d'explorer des données urbaines sur les communes en France, de visualiser des graphiques et de réaliser des prédictions à l'aide d'un **modèle d'apprentissage automatique** (Machine Learning) pour favoriser une prise de décision éclairée. L'objectif principal du projet Urban PREDICT est de répondre aux besoins des décideurs locaux, des urbanistes et des promoteurs immobiliers en leur offrant une application intuitive et performante. Le projet repose sur une **architecture client-serveur** pour assurer une expérience utilisateur fluide et des fonctionnalités avancées. Le **front-end** a été développé avec HTML/CSS et JavaScript, offrant une interface utilisateur **dynamique et intuitive**, tandis que le **back-end**, construit avec Python, gère la logique métier et les échanges de données via une **API REST**. La gestion des données s'appuie sur une **base de données relationnelle** PostgreSQL, garantissant une structuration efficace et une manipulation fluide des informations urbaines. L'intégration du modèle de Machine Learning a été une composante clé de ce projet. Développé en Python avec des bibliothèques telles que scikit-learn et pandas, le modèle analyse les données urbaines pour prédire l'état des documents d'urbanisme. L'ensemble du projet a été déployé sur le service cloud Render, offrant une disponibilité et une évolutivité optimales.

Ce dossier présente les différentes étapes du projet dans le cadre de la préparation au titre professionnel RNCP niveau 5 de développeur web et web mobile.

Contexte du projet

Expression des besoins du projet

Le projet visait à développer l'application web Urban PREDICT, dédiée à l'analyse urbaine. L'objectif principal du projet Urban PREDICT était de répondre aux besoins des décideurs locaux, des urbanistes et des promoteurs immobiliers en leur offrant une application intuitive et performante. Cette application web permet de consulter les données urbaines sur les communes en France, de visualiser des graphiques et de réaliser des prédictions à l'aide d'un modèle Machine Learning pour favoriser une prise de décision éclairée et une planification urbaine plus efficace et durable.

Contraintes du projet

Le développement de cette application s'est inscrit dans le cadre d'un projet de fin de formation, avec une durée limitée à trois mois, nécessitant une planification rigoureuse et une gestion efficace des priorités.

L'intégration du modèle de Machine Learning a représenté un défi, nécessitant un apprentissage approfondi des bibliothèques dédiées et une optimisation des performances du modèle pour garantir des temps de réponse acceptables.

Enfin, l'hébergement de l'application web sur un service cloud a soulevé des limites financières, nécessitant de choisir une solution adaptée tout en garantissant sa disponibilité.

Les livrables attendus

Le projet devait aboutir à plusieurs livrables :

- Une **application web fonctionnelle** avec une interface utilisateur permettant d'explorer les données urbaines.
- Une base de données relationnelle pour stocker toutes les données urbaines.
- L'intégration du modèle de Machine Learning pour prédire l'état des documents d'urbanisme.
- Le déploiement de l'application web sur un service cloud.

Gestion de projet

Planning et organisation

Durant ce projet, une méthode **Agile** a été utilisée pour planifier le développement de l'application web. La méthode Agile a favorisé une approche itérative, collaborative et flexible. Chaque itération a permis d'obtenir une version fonctionnelle et testable de l'application web. Ces versions ont été améliorées grâce à des réunions fréquentes durant lesquelles les problèmes rencontrés ont pu être résolus et de nouvelles priorités ont été planifiées. La flexibilité de cette méthode a permis de répondre rapidement aux imprévus et d'améliorer le livrable en continu.

Un suivi hebdomadaire des tâches a été réalisé via Trello, avec un système de tableaux Kanban permettant de classer les tâches par statut (Proposed, Approved, In progress, Dev complete, Tested et Deployed). Cela a facilité l'adaptation aux imprévus ainsi que la priorisation et l'avancement du développement.

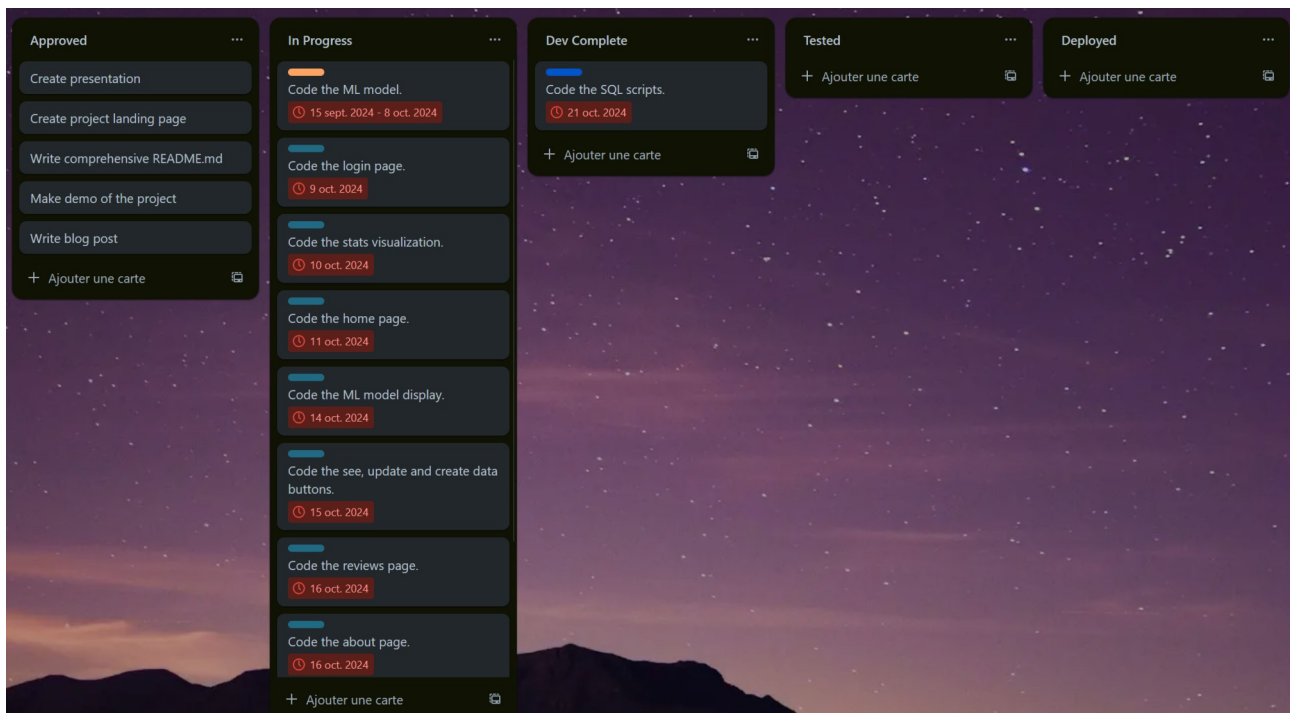


Figure 1 : Suivi des tâches via Trello

Le développement s'est fait avec Visual Studio Code, un éditeur de code léger et performant, permettant d'intégrer des extensions utiles pour le développement en JavaScript, Python et SQL, ainsi que pour la gestion des requêtes API.

Concernant le suivi des versions, Git a été utilisé pour gérer les évolutions du code en local. Le code a été stocké sur un dépôt **GitHub**, où un suivi des commits (message de la modification) et des push (envoi de la modification sur le dépôt distant) a permis de maintenir un historique clair des évolutions du projet.

Environnement humain

Le projet a été développé en autonomie mais plusieurs contributeurs ont participé à sa réalisation. Le sujet du projet a été proposé et suivi par le directeur d'Holberton School Bordeaux. Le Software Engineer d'Holberton School Bordeaux a validé le produit minimum viable et a apporté un soutien technique. Le département d'urbanisme de la Mairie de Libourne a été consulté sur les spécificités d'un plan local d'urbanisme. Enfin, les étudiants d'Holberton School ont également contribué à ce projet durant des échanges en « peer-learning », favorisant ainsi l'apprentissage en continu.

Environnement technique

Front-end : Pour l'interface utilisateur, j'ai opté pour HTML, CSS et JavaScript sans frameworks supplémentaires. Cette approche m'a permis d'approfondir mes connaissances sur JavaScript en me concentrant sur ses fonctionnalités brutes. La simplicité de HTML et de CSS m'a permis de garder le code léger et rapide tout en personnalisant entièrement les styles.

Back-end : côté serveur, j'ai choisi Python avec le framework Flask pour sa simplicité et sa légèreté dans la gestion de la création d'API. Cette solution m'a permis de mettre en place rapidement une architecture back-end fonctionnelle et évolutive, idéale pour les besoins de mon projet.

Machine Learning : Pour le composant de Machine Learning, j'ai utilisé plusieurs bibliothèques en Python telles que Scikit-learn, XGboost et Catboost permettant d'utiliser des algorithmes de classification. J'ai également utilisé Pandas et NumPy pour structurer et prétraiter les données. Enfin, Matplotlib et Seaborn ont été utilisées pour analyser et comprendre les tendances des données.

Base de données : pour la gestion du stockage des données, j'ai choisi PostgreSQL, une base de données relationnelle robuste et performante. Elle permet une gestion efficace des ensembles de données volumineux tout en offrant des performances optimisées lors des requêtes complexes.

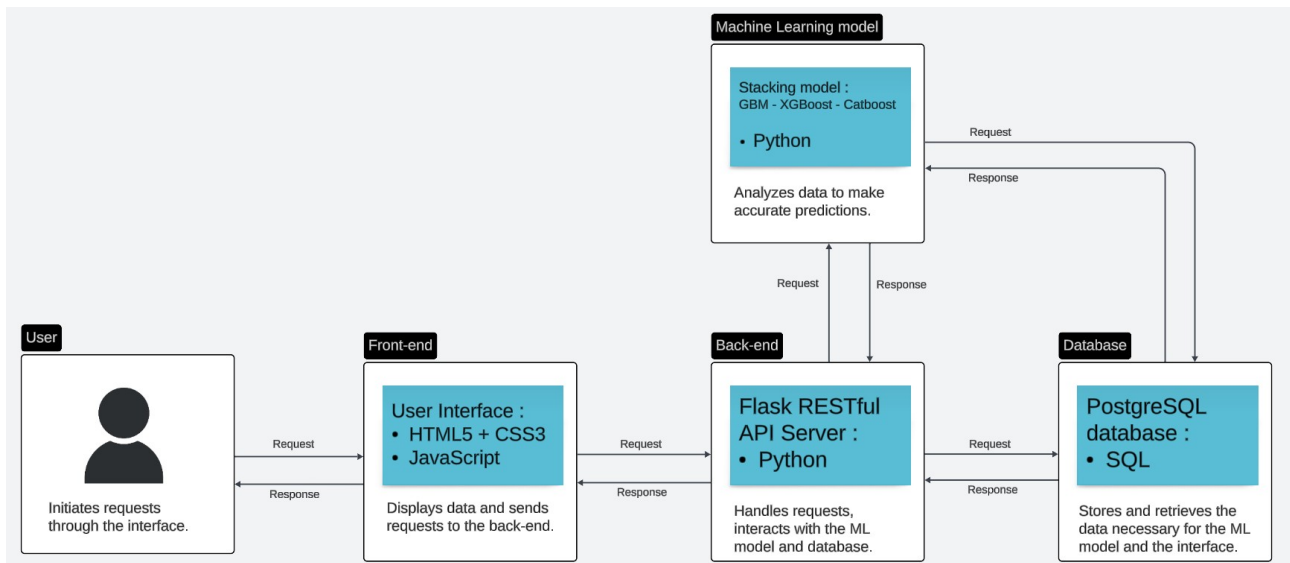


Figure 2 : Architecture et technologies

Objectifs de qualité

Pour les livrables attendus, plusieurs objectifs de qualité ont été respectés :

- L'interface utilisateur doit être pensée pour être simple, intuitive, **accessible**.
- L'interface utilisateur doit traiter les requêtes de façon dynamique.
- L'affichage de l'application web doit s'adapter au type d'équipement, y compris les équipements mobiles.
- L'affichage des résultats du modèle de Machine Learning doit être dynamique, avoir un temps de réponse adapté et des performances optimisées pour garantir la meilleure expérience utilisateur.
- La vérification du bon fonctionnement de l'application se fera via des **tests unitaires** et fonctionnels.
- Le code doit être structuré et documenté pour faciliter la maintenance et l'évolution du projet.

Réalisations côté front-end

Création de la maquette de l'interface utilisateur

Avant de commencer à coder l'interface utilisateur, un maquettage de l'interface a été réalisé avec **Balsamiq** afin de structurer l'enchaînement des pages et d'avoir un aperçu de l'application web.

Le contenu des pages web a évolué au cours du projet pour s'adapter aux demandes formulées par le staff d'Holberton school.

Les maquettes sont visibles en annexe.

Schéma d'enchaînement des maquettes

Initialement, l'utilisateur commençait sur la page d'accueil, où il devait s'authentifier pour naviguer sur l'application. Une fois authentifié, l'utilisateur pouvait naviguer dans l'application grâce aux boutons dans la barre de navigation.

Le schéma ci-dessous illustre un scénario de navigation pour comprendre l'enchaînement des maquettes.

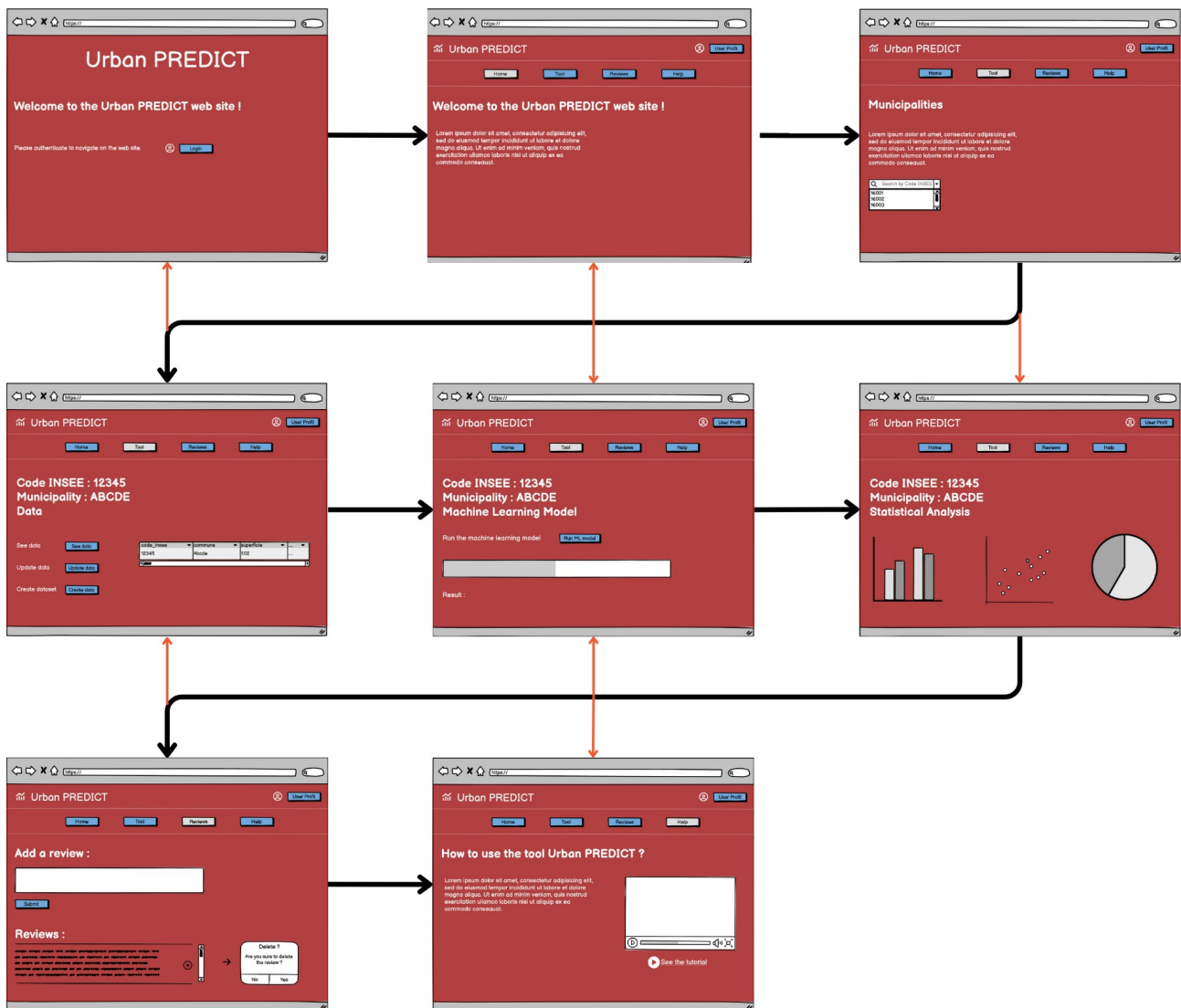


Figure 3 : Enchaînement des maquettes

Arborescence de l'application web

L'arborescence finale de l'application web est schématisée ci-dessous :

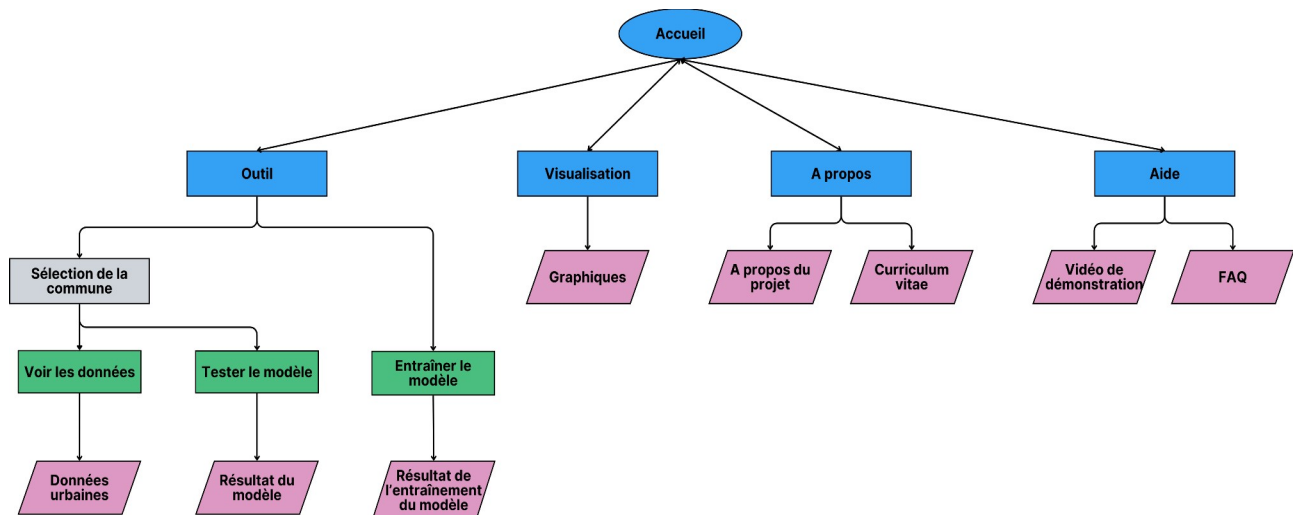


Figure 4 : Arborescence de l'application web

L'application Urban PREDICT comporte cinq pages web : Accueil, Outil, Visualisation, A propos et Aide.

La page « Outil » permet aux utilisateurs de consulter les données urbaines des communes en renseignant le code INSEE de la commune dans un champ de saisie. Elle intègre également le modèle de Machine Learning avec lequel l'utilisateur a la possibilité de tester et d'entraîner le modèle en cliquant sur un bouton.

La page « Visualisation » permet de visualiser des graphiques liés à l'ensemble des données urbaines et des résultats de l'analyse exploratoire des données.

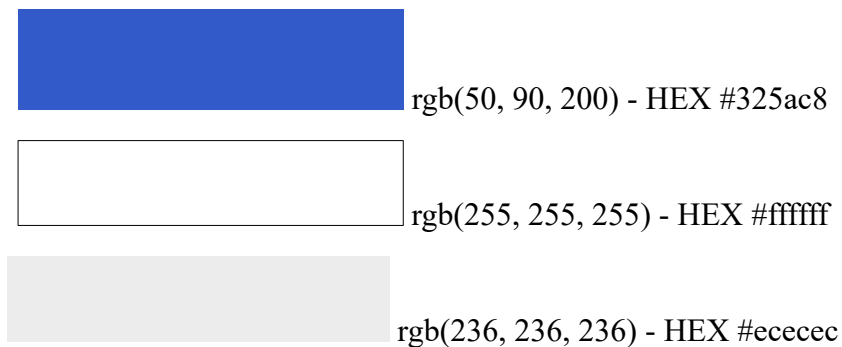
La page « A propos » présente le contexte du projet, ses objectifs et mon Curriculum Vitae.

La page « Aide » permet à l'utilisateur de regarder une vidéo de démonstration et de consulter une foire aux questions pour en apprendre plus dans le domaine de l'urbanisme et du Machine Learning.

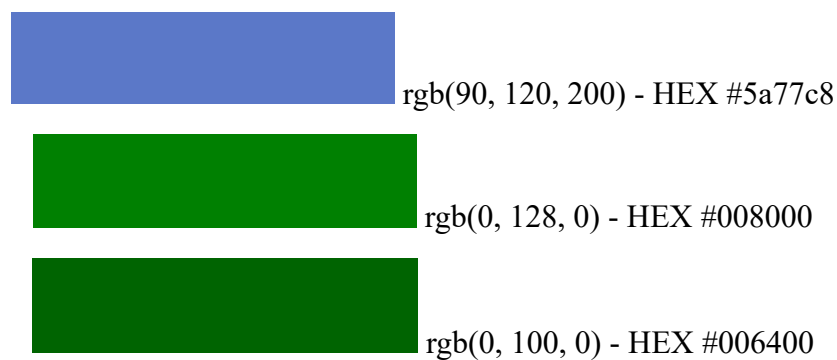
Charte graphique

Le design de l'interface utilisateur a été repensé. Les couleurs principales, axées sur des nuances de bleu, de gris et de blanc, ont permis d'assurer un bon contraste pour l'accessibilité et faciliter le confort visuel et la lisibilité.

Figure 5 : Charte graphique



Le nombre de boutons cliquables, de couleur bleue dans la barre de navigation et vert dans le corps des pages, est limité pour garantir une identification rapide et un choix intuitif.



La typographie choisie, Arial, Helvetica, sans-serif, de couleur noire, a été privilégiée pour sa simplicité et sa lisibilité, même sur des écrans de petite taille, et sa compatibilité avec la plupart des navigateurs.



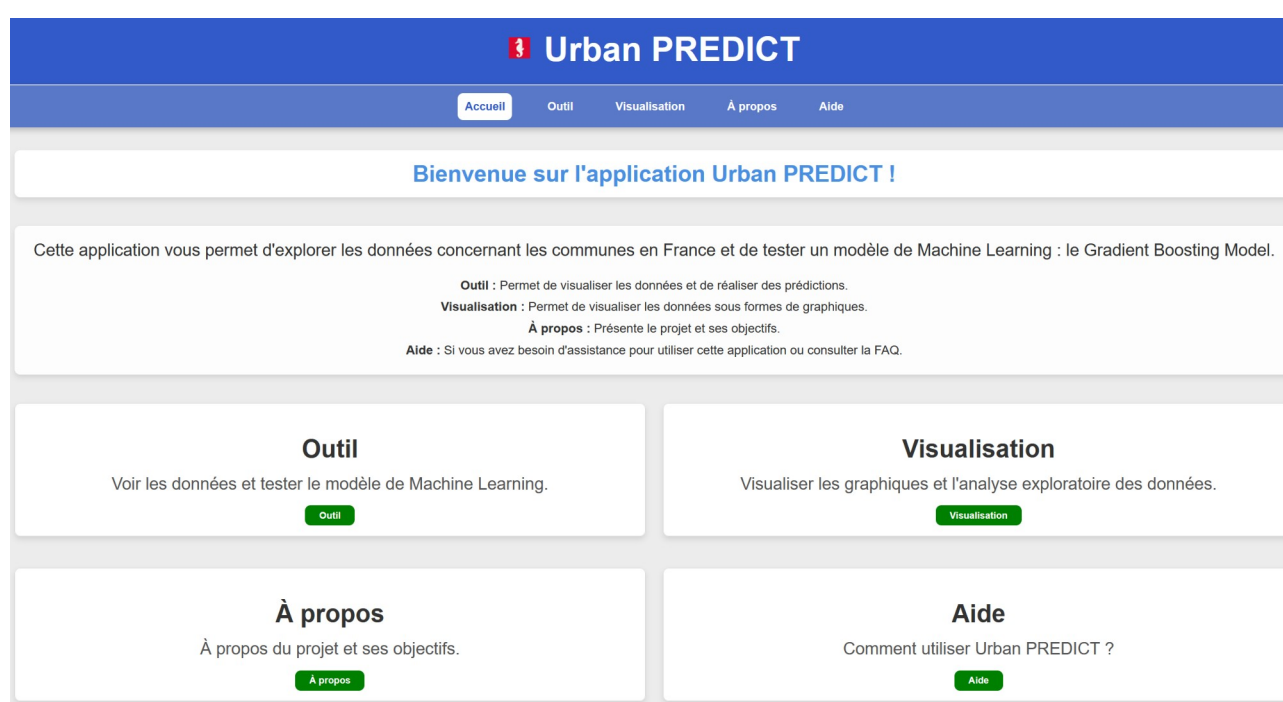
Pour le logo, j'ai utilisé le logo d'Holberton school pour symboliser ce projet de formation.



Capture d'écran de l'interface utilisateur

Certaines maquettes ont été modifiées ou supprimées pour assurer une cohérence entre le besoin de l'utilisateur et les fonctionnalités de l'application. Ainsi, la page de connexion pour s'authentifier a été supprimée pour rendre l'expérience utilisateur plus fluide. La fonctionnalité qui permettait aux utilisateurs d'alimenter la base de données a été supprimée pour se concentrer sur l'interaction avec le modèle de Machine Learning. Une page commentaires a également été supprimée.

Voici une capture d'écran de la version finale de la page d'accueil de l'application web :



Les captures d'écran des autres pages web sont visibles en annexes.

Extraits de code d'interfaces utilisateur

Le langage de présentation Hyper Text Markup Language (HTML) a été utilisé pour réaliser l'interface utilisateur. Le bloc de code ci-dessous permet à un utilisateur de rechercher, sur la page Outil, une commune en renseignant son Code INSEE dans un champ de saisie et d'afficher les données associées.

```
<!-- Main content -->
<main>
  <section>
    <!-- Title of the tool page -->
    <div>
      <h2 id="title-main-block">Outil</h2>
    </div>
  </section>

  <div class="tool-container">
    <!-- Data section -->
    <section class="data-section">
      <!-- Title and image for the data section -->
      <div id="data-card">
        <div>
          <h2>Données</h2>
        </div>
        
      </div>
      <!-- Step for Selecting a municipality -->
      <div id="stepOneDataDisplay">
        <h3>1. Sélectionnez une commune.</h3>
      </div>
      <!-- Input field for searching municipalities by INSEE code -->
      <div id="municipalities-search-field">
        <label for="code_insee"><h4>Entrez un Code INSEE :</h4></label>
        <input type="text" id="code_insee" name="code_insee" placeholder="Entrez un Code INSEE" required>
        <button id="displayDatabutton">Voir les données</button>
      </div>
      <!-- Container to display retrieved data -->
      <p id="data-output"></p>
    </section>
  </div>
</main>
```

Figure 6 : Extrait de code HTML de l'interface utilisateur

Le langage Cascading Style Sheets (CSS) a été utilisé pour implémenter le design de l'interface. Le bloc de code ci-dessous utilise une media query pour adapter l'affichage de l'interface lorsque la largeur de l'écran est inférieure ou égale à 768 pixels, s'adaptant ainsi aux équipements mobiles. La barre de navigation, l'espace entre les boutons et le titre principal s'ajustent automatiquement.

```

/* Responsive styles for screens smaller than 768px */
@media (max-width: 768px) {
  header {
    flex-direction: row; /* Stack elements vertically */
    height: 100px;
    padding: 10px 20px; /* Reduce padding */
  }

  .header-center h1 {
    font-size: 1.8rem; /* Reduce title size */
  }

  .main-logo {
    width: 25px; /* Smaller logo size */
    height: auto;
  }

  .navigation-bar {
    padding: 10px 0; /* Reduce vertical padding */
  }

  nav ul {
    flex-wrap: wrap; /* Allow wrapping on small screens */
  }

  nav a {
    font-size: 1rem;
    padding: 8px 16px; /* Adjust padding for small screens */
  }
}

```

Figure 7 : Extrait de code CSS de l'interface utilisateur

Le langage JavaScript a été utilisé pour rendre les interactions dynamiques avec l'**API REST**. Le bloc de code ci-dessous décrit la fonction asynchrone `runModel()`, utilisée pour exécuter un modèle de Machine Learning en fonction d'un Code INSEE saisi par l'utilisateur.

Tout d'abord, elle récupère et nettoie la valeur du champ `codeInseeInput`. Si aucun code n'est renseigné, un message d'erreur est affiché dans l'élément `run-output`, et la fonction s'arrête (`return`).

Ensuite, une requête POST est envoyée à l'endpoint `/data`, contenant l'action "run" ainsi que le Code INSEE en format JSON. À la réception de la réponse, les données sont converties en JSON et analysées :

- Si une erreur est retournée par le serveur (`data.error`), celle-ci est affichée dans `run-output`.
- Sinon, les résultats de la prédiction sont affichés sous forme de texte structuré avec le Code INSEE et la valeur prédite par le modèle.

En cas d'erreur réseau ou d'un problème technique, la fonction capture l'exception (catch) et informe l'utilisateur via un message d'erreur spécifique.

```
// Function to run predictions
async function runModel() {
  const codeInsee = codeInseeInput.value.trim(); // Get and trim the INSEE code input

  if (!codeInsee) { // Validate input
    document.getElementById('run-output').innerHTML = `<p>Erreur code_insee : le code INSEE est requis</p>`;
    return;
  }

  try {
    const response = await fetch('/data', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({ action: 'run', code_insee: codeInsee }) // Send 'run' action and INSEE code
    });

    const data = await response.json();
    if (data.error) { // Handle prediction errors
      document.getElementById('run-output').innerHTML = `<p>Erreur : ${data.error}</p>`;
    } else { // Display prediction results
      let runoutput = `<strong>Code INSEE : </strong> ${data.code_insee}</br>`;
      runoutput += `<strong>Résultat de la prédiction : </strong> ${data.prediction}</br>`;
      document.getElementById('run-output').innerHTML = runoutput;
    }
  } catch (error) { // Handle network errors
    console.error('Erreur :', error);
    document.getElementById('run-output').innerHTML = `<p>Erreur catch : ${error.message}</p>`;
  }
}
```

Figure 8 : Extrait de code JavaScript de l'interface utilisateur

Réalisations côté back-end

Base de données

L'application repose sur une base de données relationnelle PostgreSQL, garantissant une gestion efficace du stockage et de la récupération des données en réponse aux requêtes des utilisateurs. Les données exploitées proviennent de data.gouv.fr, une source ouverte et publique, assurant une transparence et une accessibilité optimales.

La base de données, nommée UrbanPREDICT_db, est structurée selon un modèle relationnel défini à l'aide d'un **schéma conceptuel** et **physique**. Cette base se compose de deux tables principales : « municipality », qui stocke les communes, et « features », qui contient les caractéristiques associées à ces municipalités.

L'organisation des attributs suit la convention snake_case afin d'assurer une meilleure lisibilité et maintenabilité du code.

Schémas de base de données

La table « municipality » a été créée pour stocker les informations relatives aux communes. Cette table contient l'identifiant de la commune (code_insee), le nom (nom_commune) et le département (departement).

La table « features » contient les informations urbaines associées aux communes comme des informations administratives (collectivite_porteuse, siren_epci, code_etat), l'état du document d'urbanisme de la commune (etat_commune, etat_detaille), des données sur la planification urbaine (prescription_du, approbation_du, executable_du), ainsi que des indicateurs démographiques (population_municipale, population_totale) et géographiques (superficie_insee).

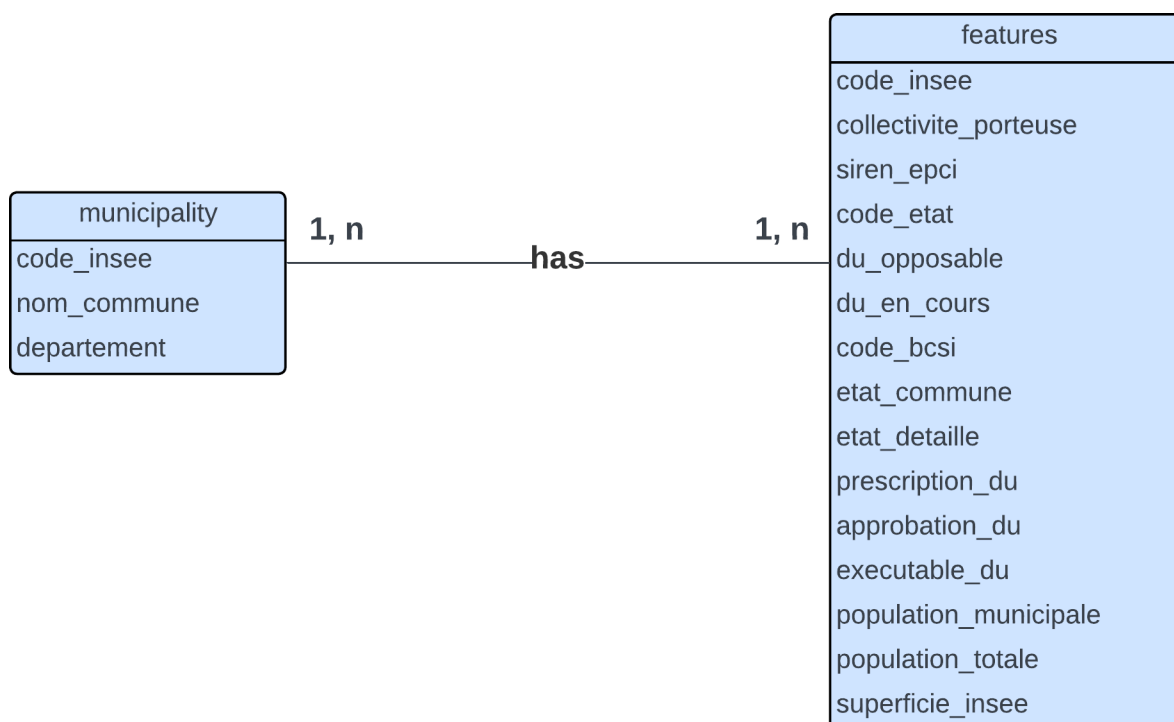


Figure 9 : Schéma conceptuel de la base de données

Chaque commune peut avoir une ou plusieurs caractéristiques. Chaque caractéristique peut avoir une ou plusieurs communes.

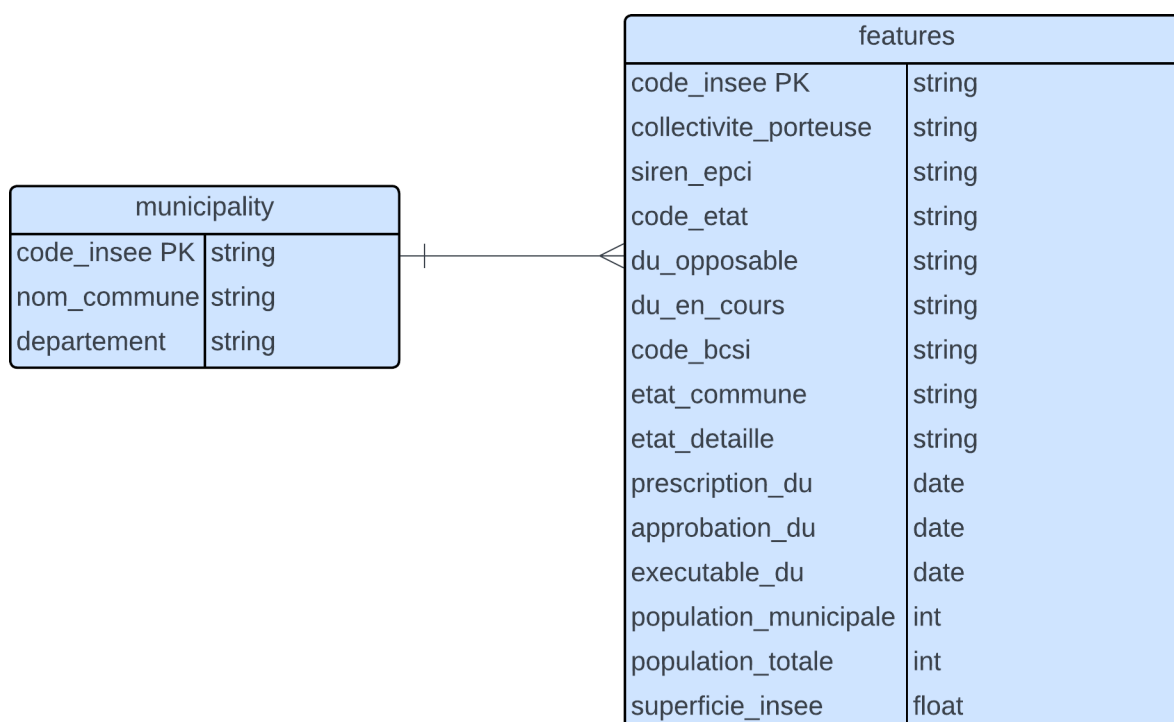


Figure 10 : Schéma physique de la base de données

Le code INSEE sert de clé primaire (PK) dans « municipality » et de clé étrangère (FK) dans « features » pour identifier les enregistrements, assurant l'intégrité des données.

Script de création de la base de données

Le script SQL implémenté ci-dessous permet de créer la base de données UrbanPREDICT_db :

```
-- Script that creates the database UrbanPREDICT_db
CREATE DATABASE UrbanPREDICT_db;

-- Connection to the database
\c UrbanPREDICT_db;

-- Script that creates the table municipality
CREATE TABLE IF NOT EXISTS municipality (
    code_insee VARCHAR(5) PRIMARY KEY NOT NULL,
    nom_commune VARCHAR(100) NOT NULL,
    departement VARCHAR(3) NOT NULL
);

-- Script that creates the table features
CREATE TABLE IF NOT EXISTS features (
    code_insee VARCHAR(5) NOT NULL,
    collectivite_porteuse VARCHAR(100),
    siren_epci VARCHAR(9),
    code_etat VARCHAR(2),
    du_opposable VARCHAR(5),
    du_en_cours VARCHAR(5),
    code_bcsi VARCHAR(4),
    etat_commune VARCHAR(100),
    etat_detaille VARCHAR(100),
    prescription_du DATE,
    approbation_du DATE,
    executable_du DATE,
    population_municipale INT,
    population_totale INT,
    superficie_insee FLOAT,
    PRIMARY KEY (code_insee),
    CONSTRAINT fk_municipality FOREIGN KEY (code_insee) REFERENCES municipality(code_insee) ON DELETE CASCADE
);
```

Figure 11 : Script SQL pour la création de la base de données

Extrait de code de composant métier

Sur la page Outil, l'utilisateur a la possibilité d'entraîner le modèle de Machine Learning en cliquant sur un bouton **Entraîner le modèle**. Ce clic déclenche l'exécution d'un script contenant plusieurs fonctions et notamment celle d'entraîner les modèles spécifiquement XGBoost et CatBoost, puis le modèle final de Gradient Boosting.

```
# Function to train the models XGBoost and Catboost
def train_models(X_train_num, X_train_cat, y_train):
    # Configure and train XGBoost
    xgb_model = xgb.XGBClassifier(max_depth=1, learning_rate=0.01, n_estimators=10, eval_metric='logloss', n_jobs=1)
    xgb_model.fit(X_train_num, y_train)
    xgb_preds_train = xgb_model.predict_proba(X_train_num)[: , 1]

    # Configure and train CatBoost
    catboost_model = cb.CatBoostClassifier(depth=1, learning_rate=0.01, iterations=10, verbose=0, cat_features=np.arange(X_train_cat.shape[1]), thread_count=1)
    catboost_model.fit(X_train_cat, y_train)
    catboost_preds_train = catboost_model.predict_proba(X_train_cat)[: , 1]

    return xgb_model, catboost_model, xgb_preds_train, catboost_preds_train
```

Figure 12 : Extrait de code Python de composant métier

La fonction `train_models` entraîne deux modèles de classification, XGBoost et CatBoost, à partir de données numériques (`X_train_num`) et catégorielles (`X_train_cat`). Les modèles XGBoost et CatBoost sont configurés avec plusieurs hyperparamètres, qui sont définis manuellement et influencent le comportement et les performances des modèles. Après l'entraînement, chaque modèle génère des probabilités d'appartenance à une classe spécifique pour l'état du document d'urbanisme. La fonction retourne ensuite les résultats de l'entraînement des deux modèles ainsi que leurs prédictions respectives sur l'ensemble des données d'entraînement. Ces résultats alimenteront ensuite un modèle final de Gradient Boosting pour obtenir la prédiction finale.

```
# Generate predictions using the loaded models
xgb_preds = xgb_model.predict_proba(quantitative_features)[: , 1]
catboost_preds = catboost_model.predict_proba(categorical_features)[: , 1]

# Stack predictions from the xgboost and catboost models
stacked_features = np.column_stack((xgb_preds, catboost_preds))
```

Cette approche permet d'exploiter les forces de chaque algorithme sur des types de données différents, améliorant ainsi les performances globales.

L'entraînement du modèle étant relativement long, cela dégradait l'expérience utilisateur. Pour y remédier, une solution de pré-entraînement a été mise en place, permettant de **tester le modèle** et d'obtenir les prédictions en seulement quelques secondes et d'améliorer ainsi l'expérience utilisateur.

```
# Run the prediction and print the JSON output
if code_insee:
    output = run_prediction(code_insee)
    print(output)
else:
    # Print an error if the INSEE code is missing
    print(json.dumps({"error": "Code INSEE manquant"}))
```

Extrait de code de composant d'accès aux données

Pour accéder aux données d'une commune à partir de la base de données PostgreSQL, un composant d'accès aux données, Psycopg2, a été utilisé. La fonction `get_municipality(code_insee)` établit une connexion à la base de données. Une requête SQL est ensuite exécutée pour extraire les données souhaitées en reliant les tables « municipality » et « features ». Enfin, la connexion est fermée après l'extraction des résultats, optimisant ainsi la gestion des ressources et la sécurité.

```
def get_municipality(code_insee):
    try:
        # Retrieve the database URL from environment variables
        DATABASE_URL = os.environ.get('DATABASE_URL')
        if DATABASE_URL is None:
            # Raise an error if the DATABASE_URL environment variable is not set
            raise ValueError("La variable d'environnement DATABASE_URL n'est pas définie.")

        # Establish a connection to the PostgreSQL database
        conn = psycopg2.connect(DATABASE_URL)
        cursor = conn.cursor()

        # Define the SQL query to fetch the data
        query = '''
            SELECT m.nom_commune, m.departement,
                   f.population_municipale, f.population_totale, f.superficie_insee,
                   f.collectivite_porteuse, f."siren_epci", f.code_etat, f."du_opposable",
                   f."du_en_cours", f."code_bcsi", f.etat_commune, f.etat_detaille,
                   TO_CHAR(f.prescription_du, 'DD/MM/YYYY') AS prescription_du,
                   TO_CHAR(f.approbation_du, 'DD/MM/YYYY') AS approbation_du,
                   TO_CHAR(f.executoire_du, 'DD/MM/YYYY') AS executable_du
            FROM municipality m
            JOIN features f ON m.code_insee = f.code_insee
            WHERE m.code_insee = %s;
        '''

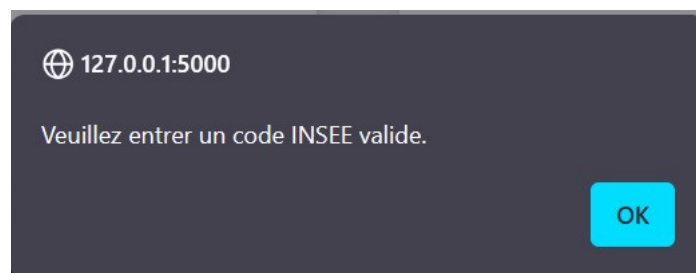
        # Execute the query with the 'code_insee' received
        cursor.execute(query, (code_insee,))
        # Fetch the first result from the query
        municipality = cursor.fetchone()
        # Close the database connection
        conn.close()
```

Figure 13 : Extrait de code Python de composant d'accès à la base de données

Ces réalisations côté front-end et back-end ont permis de développer une application complète, avec une **interface fluide** et un **côté serveur performant**. Grâce à cette organisation modulaire, l'application est évolutive et pourra être améliorée dans de futures versions.

Éléments de sécurité de l'application

L'un des éléments les plus importantes en matière de sécurité pour l'application Urban PREDICT est la validation des entrées. Sur la page web Outil, le champ de saisie du Code INSEE est validé, avant d'être envoyé côté serveur, par le code JavaScript. Par exemple, une vérification est effectuée pour s'assurer qu'un Code INSEE est bien fourni avant d'exécuter une requête.



Côté serveur, Flask utilise par défaut Jinja2, qui applique une protection contre les injections de code HTML en échappant les caractères spéciaux. Si le Code INSEE renseigné est incorrecte, un message d'erreur est affiché à l'utilisateur. Les risques liés aux entrées malveillantes sont ainsi réduits.

Concernant les autres fonctionnalités, l'application web gère les erreurs de manière proactive, en affichant des messages d'erreur à l'utilisateur sans divulguer d'informations sensibles. Ce style défensif protège contre les fuites d'informations.

L'application suit une architecture client-serveur où le front-end, la logique métier et la gestion des données sont séparés. Cette approche réduit la surface d'attaque et empêche un accès direct à la base de données depuis l'interface utilisateur.

Pour prévenir les attaques par injection SQL, l'application utilise des requêtes préparées. Cette méthode empêche l'exécution de commandes malveillantes en séparant les données des instructions SQL. Cela garantit des échanges sécurisés avec la base de données.

La solution d'hébergement utilisée offre un chiffrement SSL pour les échanges de données et une protection anti-DDOS. Ces mécanismes apportent à l'application une couche supplémentaire de sécurité contre les menaces en ligne.

Jeu d'essai

Une série de tests unitaires a été réalisée côté front-end et côté back-end pour tester la fonctionnalité des points de terminaison de l'API Flask, les méthodes associées et les réponses attendues.

Pour cela, le framework Jest a été utilisé pour le code JavaScript et le module unittest en Python pour le back-end.

Tests côté front-end

Après avoir installé les dépendances nécessaires et exécuté le script ci-dessous, le framework Jest a permis de tester le front-end :

Figure 14 : Tests unitaires

```
/**
 * @jest-environment jsdom
 */

import { render, screen, within } from "@testing-library/react";
import { fireEvent } from "@testing-library/dom";
import "@testing-library/jest-dom";

document.body.innerHTML = `
  <header>
    <div class="header-left">
      <a href="home">
        
      </a>
    </div>
    <div class="header-center">
      <h1>Urban PREDICT</h1>
    </div>
  </header>

  <nav class="navigation-bar">
    <ul class="nav-links">
      <li><a href="home" id="home-link">Accueil</a></li>
      <li><a href="data" id="data-link" class="active">Outil</a></li>
      <li><a href="visualization" id="Stats-link">Visualisation</a></li>
      <li><a href="about" id="about-link">À propos</a></li>
      <li><a href="help" id="help-link">Aide</a></li>
    </ul>
  </nav>
`
```

```

<main>
  <section>
    <h2 id="title-main-block">Outil</h2>
  </section>

  <div class="tool-container">
    <!-- Section Data -->
    <section class="data-section">
      <div id="municipalities-search-field">
        <label for="code_insee"><h4>Entrez un Code INSEE :</h4></label>
        <input type="text" id="code_insee" name="code_insee" placeholder="Entrez un Code INSEE" required>
        <button id="displayDatabutton">Voir les données</button>
      </div>
      <p id="data-output"></p>
    </section>

    <!-- Section ML -->
    <section class="ml-section">
      <button id="trainMLModelbutton">Entraîner</button>
      <p id="train-output"></p>
      <button id="runMLModelbutton">Tester</button>
      <p id="run-output"></p>
    </section>
  </div>
</main>
</script>
</html>
</test>

// Test if the navigation bar contains all links
test("La barre de navigation contient tous les liens", () => {
  const nav = screen.getByRole("navigation");
  const links = within(nav).getAllByRole("link");

  expect(links.map(link => link.textContent)).toEqual([
    "Accueil",
    "Outil",
    "Visualisation",
    "À propos",
    "Aide"
  ]);
});

// Test if the main title is correct
test("Le titre principal doit être 'Outil'", () => {
  expect(screen.getByRole("heading", { level: 2, name: "Outil" })).toBeInTheDocument();
});

// Test if the Code INSEE input field exists
test("Le champ de saisie du Code INSEE est présent", () => {
  const input = screen.getByPlaceholderText("Entrez un Code INSEE");
  expect(input).toBeInTheDocument();
  expect(input).toHaveAttribute("id", "code_insee");
});

// Test if the "Voir les données" button is present and clickable
test("Le bouton 'Voir les données' est fonctionnel", () => {
  const button = screen.getByText("Voir les données");
  expect(button).toBeInTheDocument();
  expect(button).toBeEnabled();
});

// Test if the Machine Learning buttons are displayed
test("Les boutons de Machine Learning doivent être affichés", () => {
  expect(screen.getByText("Entraîner")).toBeInTheDocument();
  expect(screen.getByText("Tester")).toBeInTheDocument();
});

// Interaction test: Check if a message appears after a click
test("Simule un clic sur 'Entraîner' et vérifie le message", () => {
  const trainButton = screen.getByText("Entraîner");
  const trainOutput = screen.getByText("", { selector: "#train-output" });

  fireEvent.click(trainButton);

  // Simulate a message in the DOM
  trainOutput.textContent = "Entraînement du modèle en cours...";
  expect(trainOutput).toHaveTextContent("Entraînement du modèle en cours...");
});

```

Les résultats obtenus ont montré que tous les tests ont été passés avec succès :

```
> test
> jest

PASS ./html.test.js
  ✓ La barre de navigation contient tous les liens (71 ms)
  ✓ Le titre principal doit être 'Outil' (13 ms)
  ✓ Le champ de saisie du Code INSEE est présent (3 ms)
  ✓ Le bouton 'Voir les données' est fonctionnel (5 ms)
  ✓ Les boutons de Machine Learning doivent être affichés (8 ms)
  ✓ Simule un clic sur 'Entraîner' et vérifie le message (9 ms)

Test Suites: 1 passed, 1 total
Tests:       6 passed, 6 total
Snapshots:   0 total
Time:        1.641 s, estimated 2 s
Ran all test suites.
```

Tests côté back-end

Le script ci-dessous a permis de réaliser un test complet du back-end en utilisant unittest, un module natif de Python :

```

import unittest
import sys
import os
from run import create_app

# Add the parent directory to the path
sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))

""" Unit test class for testing Flask application routes and functionalities """

class MyFlaskTestCase(unittest.TestCase):

    def setUp(self):
        """ A method that sets up the environment """
        self.app = create_app().test_client()
        self.app.testing = True

    def test_home_page(self):
        """ Test for the home page (route '/') """
        response = self.app.get('/')
        self.assertEqual(response.status_code, 200)
        self.assertIn(b'Accueil', response.data)

    def test_home_route(self):
        """ Test for the '/home' route """
        response = self.app.get('/home')
        self.assertEqual(response.status_code, 200)
        self.assertIn(b'Accueil', response.data)

    def test_visualization_route(self):
        """ Test for the '/visualization' route """
        response = self.app.get('/visualization')
        self.assertEqual(response.status_code, 200)
        self.assertIn(b'Visualisation', response.data)

    def test_data_post_without_action(self):
        """ Test for the '/data' route with a POST request without an action """
        response = self.app.post('/data', json={})
        self.assertEqual(response.status_code, 400)
        self.assertIn(b'Aucune action sp\\u00e9cifi\\u00e9e', response.data)

    def test_data_post_with_invalid_action(self):
        """ Test for the '/data' route with a POST request and an invalid action """
        response = self.app.post('/data', json={'action': 'invalid_action'})
        self.assertEqual(response.status_code, 400)
        self.assertIn(b'Action non reconnue', response.data)

```

```

def test_data_post_with_view_and_missing_code_insee(self):
    """ Test for the '/data' route with 'view' action but without 'code_insee' """
    response = self.app.post('/data', json={'action': 'view'})
    self.assertEqual(response.status_code, 400)
    self.assertIn(b'Code INSEE manquant', response.data)

def test_data_post_with_run_and_valid_code_insee(self):
    """ Test for the '/data' route with 'run' action and a valid 'code_insee' """
    response = self.app.post('/data', json={'action': 'run', 'code_insee': '123456'})
    self.assertEqual(response.status_code, 500)
    self.assertIn(b'{' , response.data)

def test_get_municipality_valid_code_insee(self):
    """ Test to check retrieval of a valid municipality """
    response = self.app.post('/data', json={'action': 'view', 'code_insee': '16001'})
    self.assertEqual(response.status_code, 200)
    self.assertIn(b'Bordeaux', response.data)

def test_get_municipality_invalid_code_insee(self):
    """ Test to verify handling of an incorrect 'code INSEE' """
    response = self.app.post('/data', json={'action': 'view', 'code_insee': '000000'})
    self.assertEqual(response.status_code, 404)
    self.assertIn(b'Code INSEE incorrecte !', response.data)

def test_exec_script_model(self):
    """ Test to verify the execution of the model script """
    response = self.app.post('/data', json={'action': 'train'})
    self.assertEqual(response.status_code, 200)
    self.assertIn(b'accuracy', response.data)

def test_run_model(self):
    """ Test for running the model with a valid 'code INSEE' """
    response = self.app.post('/data', json={'action': 'run', 'code_insee': '16001'})
    self.assertEqual(response.status_code, 200)
    self.assertIn(b'{' , response.data)

def test_run_model_without_code_insee(self):
    """ Test to ensure an error is returned if 'code INSEE' is missing """
    response = self.app.post('/data', json={'action': 'run'})
    self.assertEqual(response.status_code, 400)
    self.assertIn(b'Code INSEE manquant', response.data)

if __name__ == '__main__':
    unittest.main()

```

La capture d'écran ci-dessous montre que les tests ont été réalisés avec succès :

```

.....
-----
Ran 12 tests in 120.774s

OK

```

Veille sur les vulnérabilités de sécurité

Protections contre les attaques Cross-Site Scripting (XSS)

L’XSS est une vulnérabilité où un attaquant injecte du code malveillant dans une page web que d’autres utilisateurs vont ensuite exécuter. L’application doit s’assurer que toutes les entrées utilisateur sont correctement échappées avant d’être affichées sur la page web pour empêcher l’injection de scripts malveillants. Dans le cas de mon application, lorsqu’un utilisateur tape un Code INSEE ou toutes autres informations dans le champ de saisie, ces informations doivent être correctement filtrées pour éviter l’exécution de code malveillant.

Protections contre les injections SQL (SQLi)

Une attaque par injection SQL se produit lorsqu’un attaquant est capable d’injecter une commande SQL malveillante dans une requête, ce qui pourrait compromettre la sécurité de la base de données. Pour éviter cela, il est nécessaire d’utiliser des requêtes préparées qui gèrent automatiquement l’échappement des paramètres dans les requêtes SQL.

Protections contre les attaques Cross-Site Request Forgery (CSRF)

Sécurité des sessions utilisateurs : L’application ne gère pas de sessions utilisateurs. Dans le cas où un système d’authentification serait implémentée, la gestion des sessions serait sécurisée avec des techniques comme les tokens CSRF et la gestion des cookies HttpOnly afin d’empêcher un attaquant de soumettre des requêtes non autorisées en utilisant l’identité d’un utilisateur authentifié ainsi que la mise en place de timeouts de session.

Vulnérabilités de bibliothèques tierces

L’import de bibliothèques tierces pour le développement du projet peut comporter des risques pour la sécurité de l’application web. Il est nécessaire de veiller à importer les versions stables des bibliothèques et de les mettre à jour régulièrement.

Documenter le déploiement

Le déploiement de l'application web Urban PREDICT s'est fait sur le service en ligne Render.

Solution d'hébergement

Render permet d'héberger une application web sans gérer l'infrastructure sous-jacente, simplifiant ainsi la mise en ligne d'un projet. Le déploiement se fait en quelques étapes avec peu de configuration, grâce à une intégration directe avec GitHub.

La plateforme propose un certificat SSL gratuit, un domaine personnalisé, un load balancer intégré et une mise à l'échelle automatique pour ajuster les ressources selon la charge.

Cette solution d'hébergement cloud prend en charge Python et supporte aussi bien les applications back-end que les bases de données PostgreSQL. Elle permet également d'exécuter des tâches en arrière-plan. Render offre une formule d'hébergement gratuite mais avec des performances limitées par rapport aux offres payantes.

Script de déploiement

Pour un déploiement de l'application, j'ai utilisé un fichier « requirements.txt » pour gérer les dépendances nécessaires au bon fonctionnement du projet.

```
pandas
numpy
xgboost
catboost
seaborn
matplotlib
scikit-learn
flask
flask-cors
psycopg2-binary
joblib
gunicorn
flask_sqlalchemy
flask-jwt-extended
python-dotenv
```

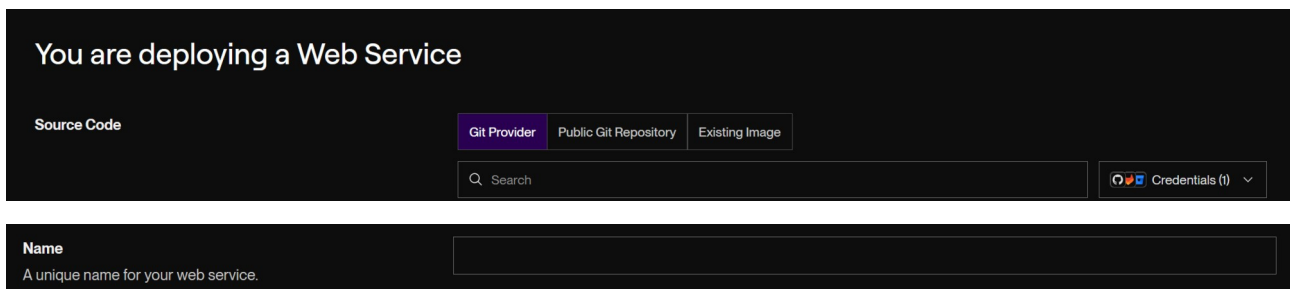
Figure 15 : Liste des dépendances dans requirements.txt

Étapes avant mise en production

Les étapes suivantes décrivent les étapes avant mise en production de l'application à partir du tableau de bord sur Render :

- Connexion au dépôt Github :

Figure 16 : Étapes avant mise en production



You are deploying a Web Service

Source Code

Git Provider | Public Git Repository | Existing Image

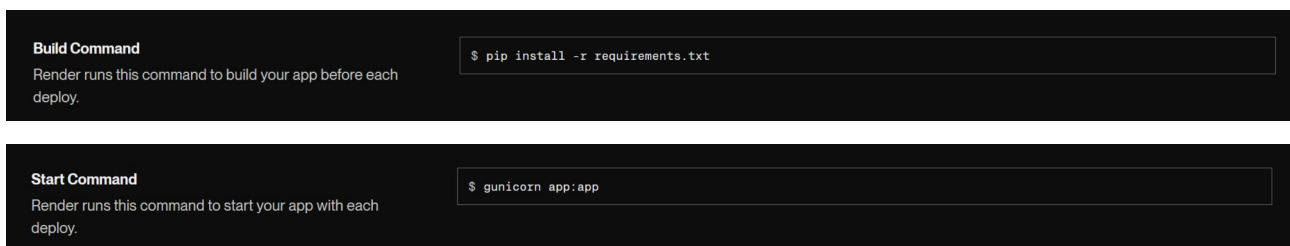
Search

Credentials (1)

Name

A unique name for your web service.

- Installation des dépendances (requirements.txt) :



Build Command

Render runs this command to build your app before each deploy.

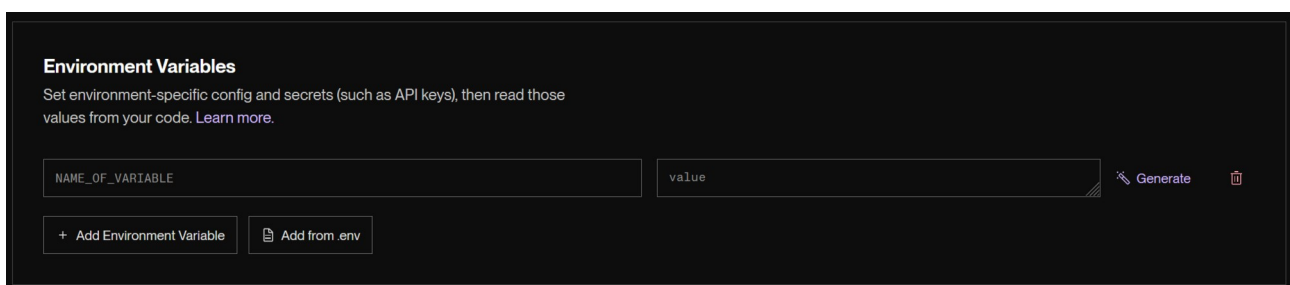
\$ pip install -r requirements.txt

Start Command

Render runs this command to start your app with each deploy.

\$ gunicorn app:app

- Configuration des variables d'environnement :



Environment Variables

Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more.](#)

NAME_OF_VARIABLE

value

+ Add Environment Variable

Add from .env

Generate

- Déploiement l'application web :



Deploy Web Service

- Création la base de données :

New PostgreSQL

[View docs](#)

Name
A unique name for your PostgreSQL instance.

example-postgresql-name

Create Database

- Configuration de la base de données :

External Database URL

.....

PSQL Command

.....

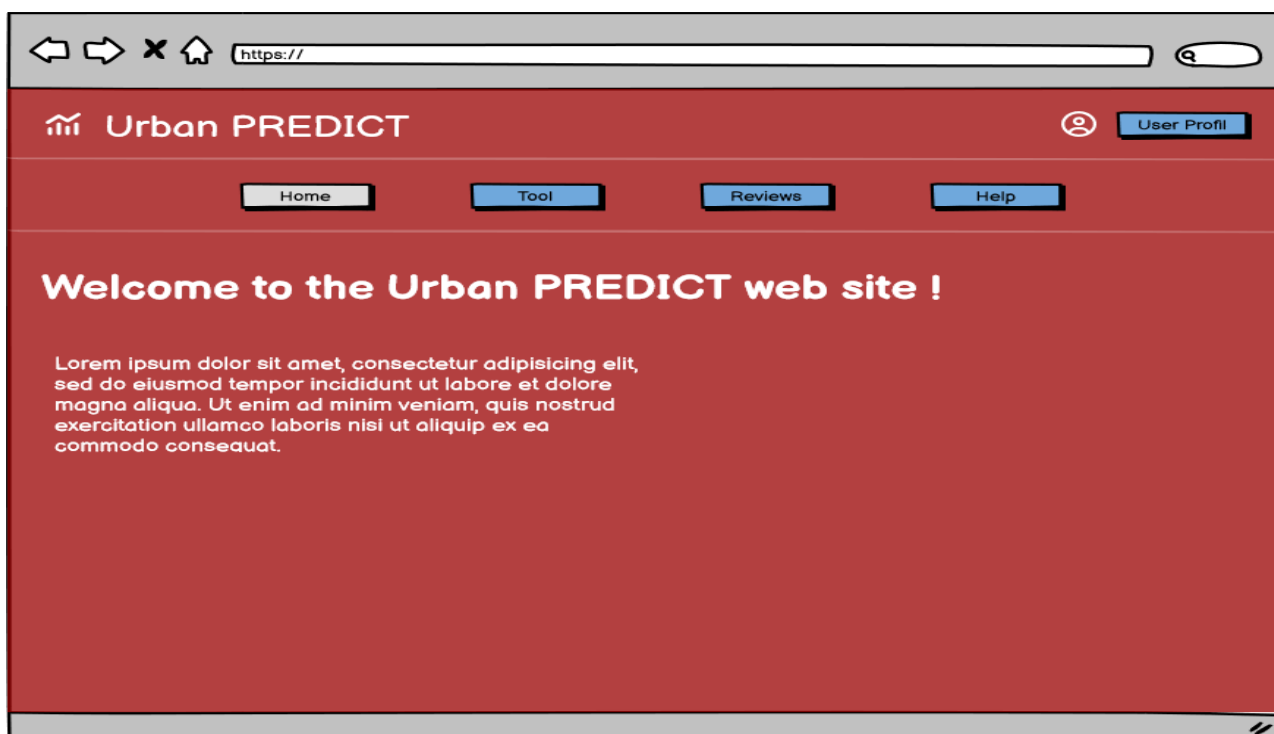
Conclusion

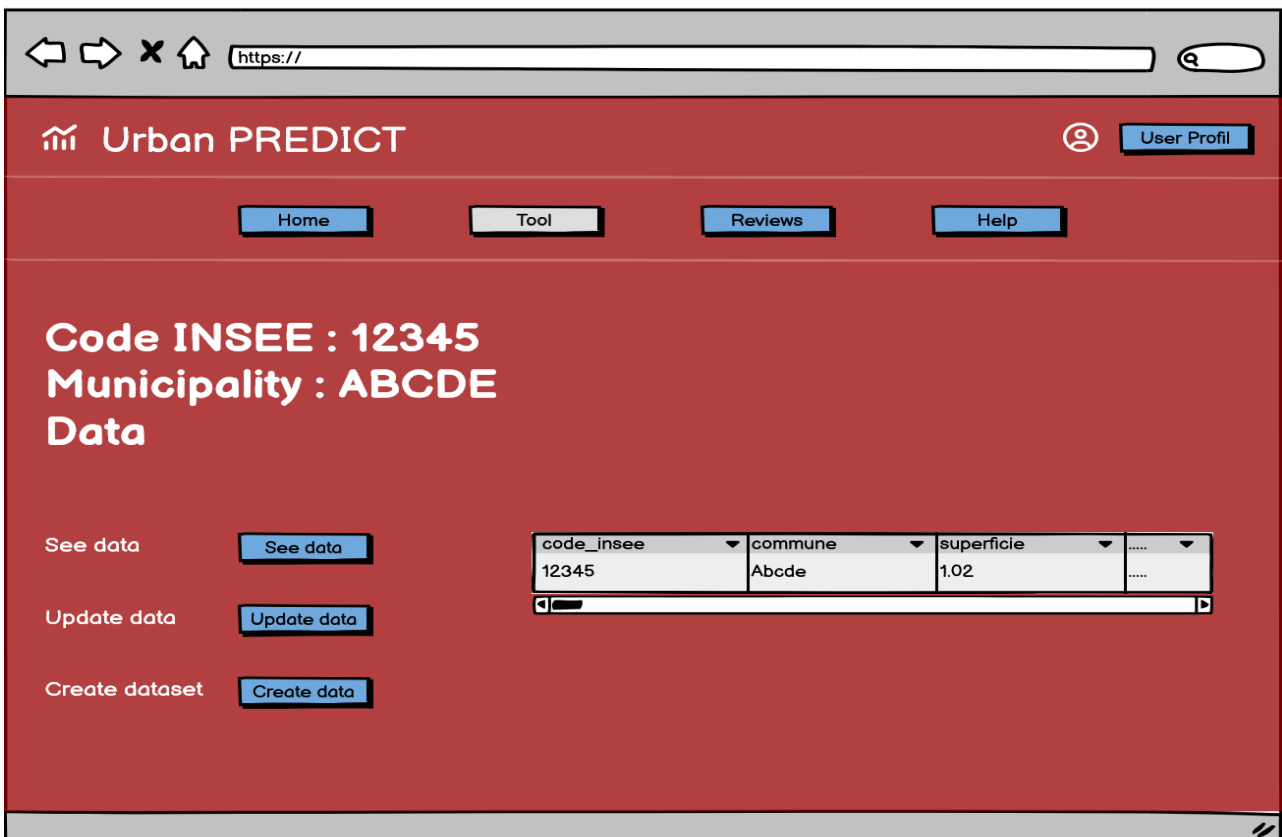
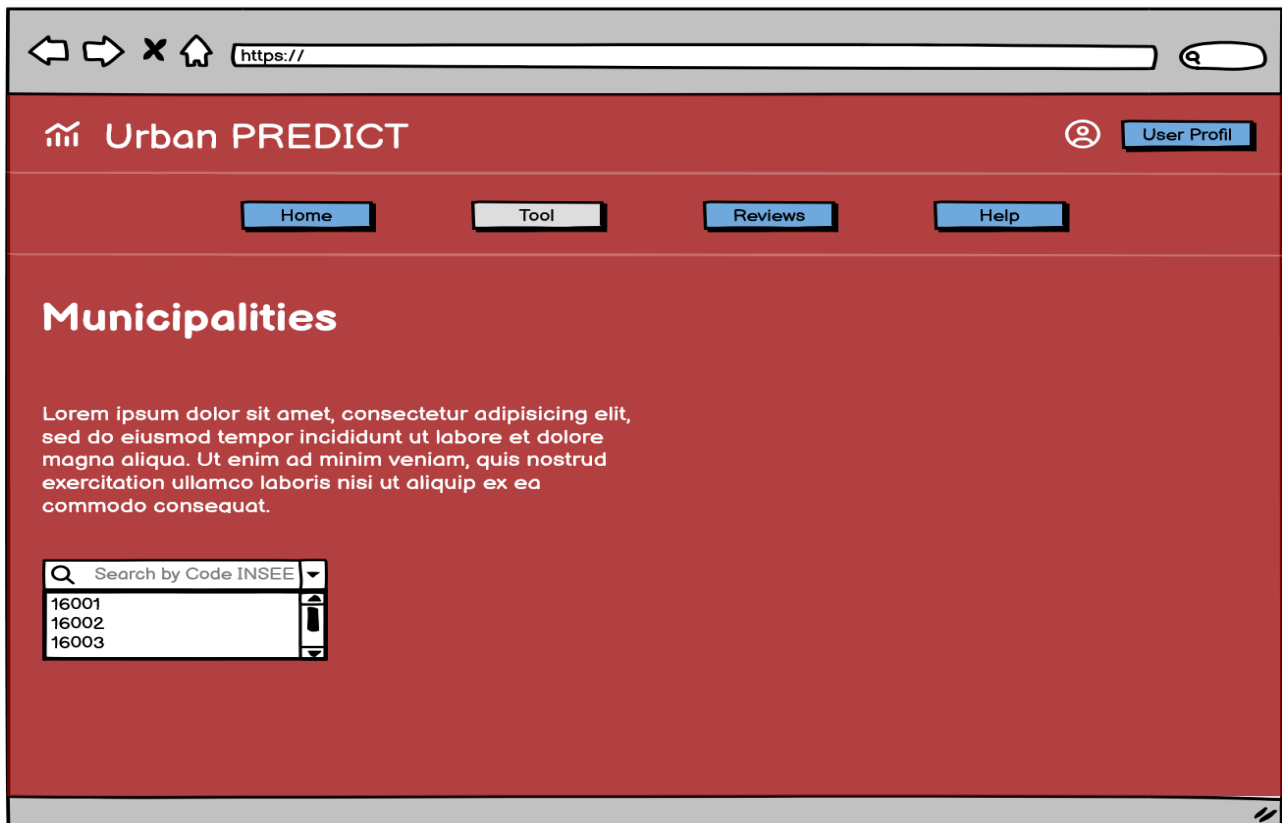
Le projet Urban PREDICT avait pour objectif de développer une application web permettant aux utilisateurs d'accéder à des données urbaines et d'utiliser un modèle de Machine Learning pour générer des prédictions sur l'état d'un document d'urbanisme. À travers ce développement, l'accent a été mis sur l'accessibilité, la performance et l'intégration fluide des différentes technologies afin d'offrir une solution fonctionnelle. Pour atteindre cet objectif, plusieurs compétences techniques ont été mobilisées : développement front-end et back-end, gestion de la base de données et déploiement d'une application web dans un environnement cloud. Chaque étape du développement m'a permis d'approfondir mes connaissances et de mettre en pratique les bonnes pratiques de programmation et d'architecture logicielle. Au-delà de l'aspect technique, le projet Urban PREDICT a également été un exercice de gestion et d'**adaptation**, renforçant mes compétences transversales. La méthode Agile a facilité la planification du projet et permis une meilleure réactivité face aux défis rencontrés. La résolution de problèmes techniques a été un élément clé, nécessitant une recherche constante de solutions et une optimisation continue. Enfin, travailler en autonomie tout en collaborant avec d'autres intervenants a exigé une **communication** efficace, une **organisation** rigoureuse et une capacité à s'adapter aux attentes des utilisateurs et aux contraintes du développement. L'application pourrait être optimisée grâce à une meilleure gestion des performances du modèle de Machine Learning, une extension des données urbaines exploitées et une automatisation de la mise à jour de la base de données.

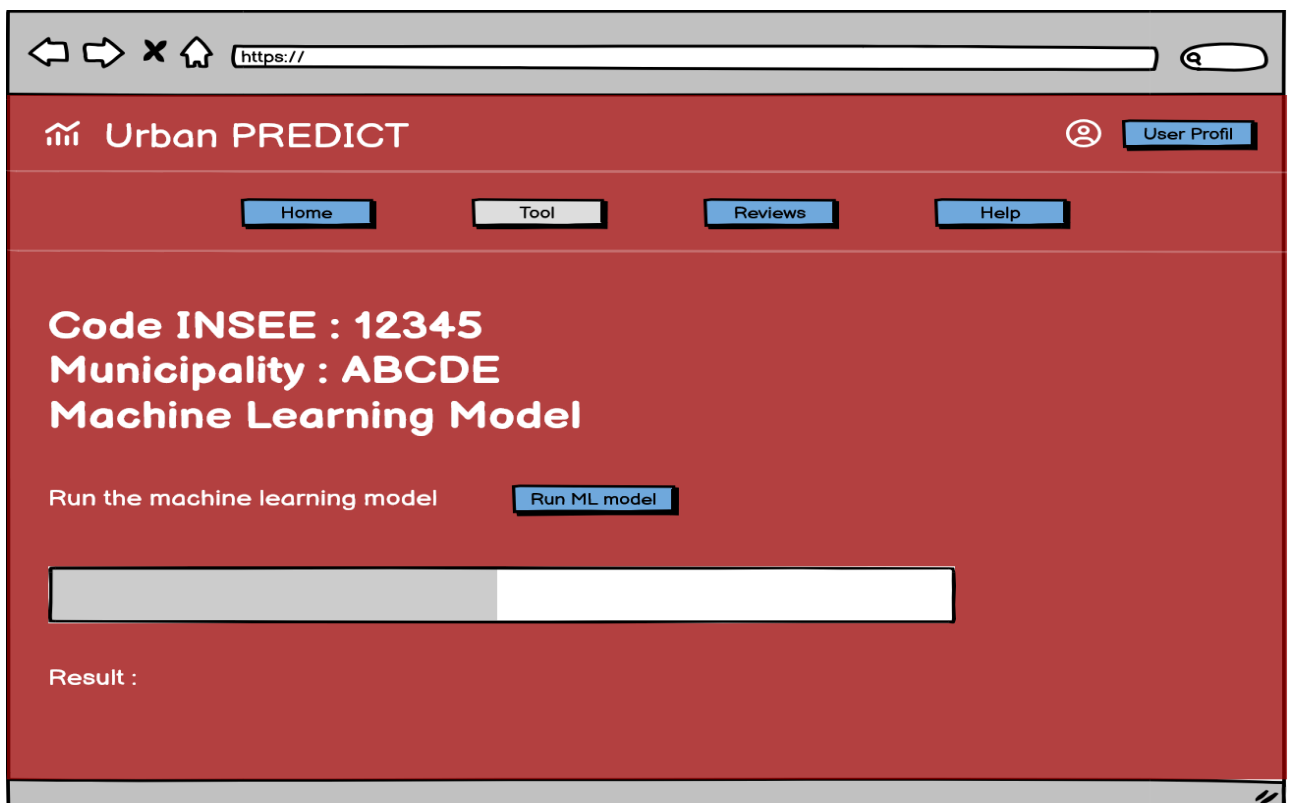
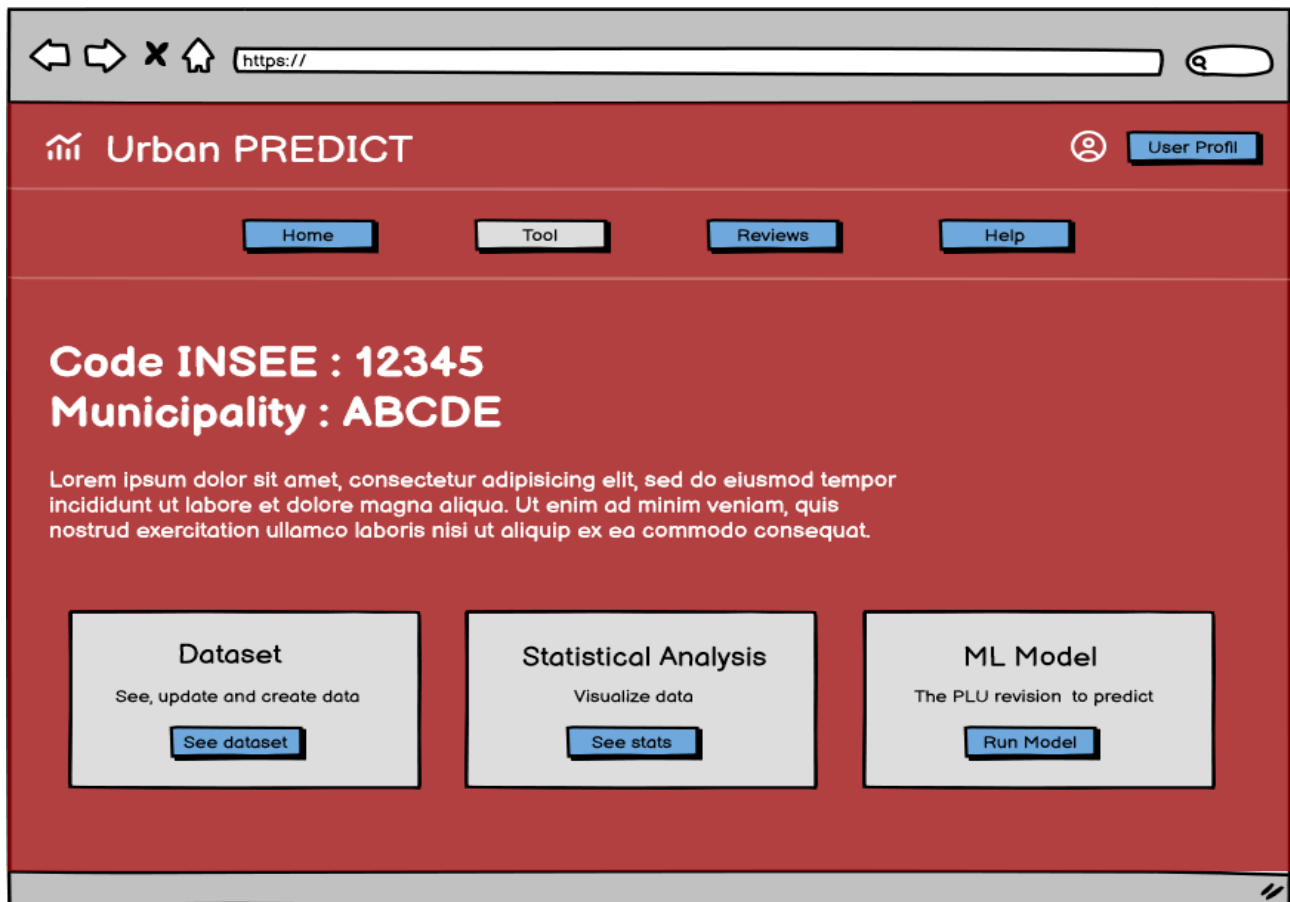
En définitive, ce projet a été une expérience enrichissante et m'a permis de développer les compétences attendues pour le titre professionnel RNCP 5 de développeur web et web mobile.

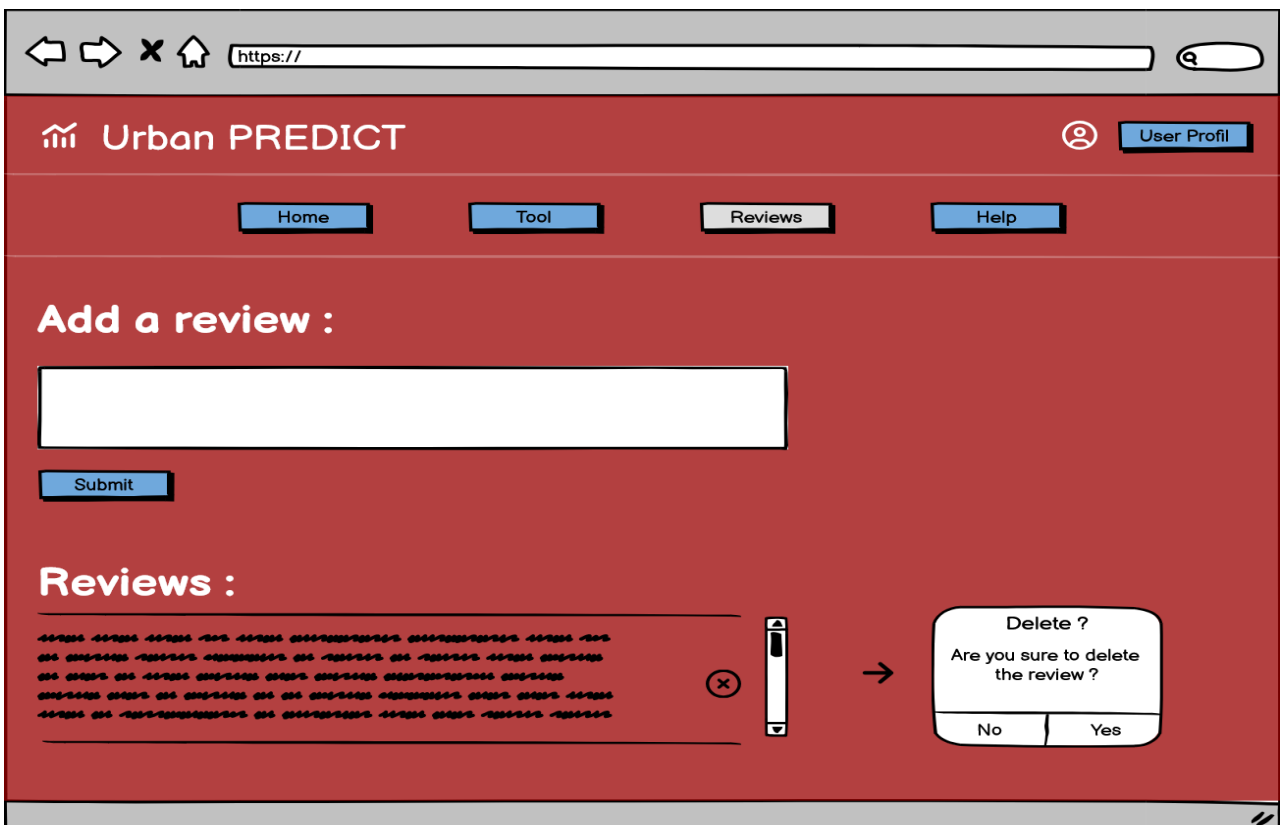
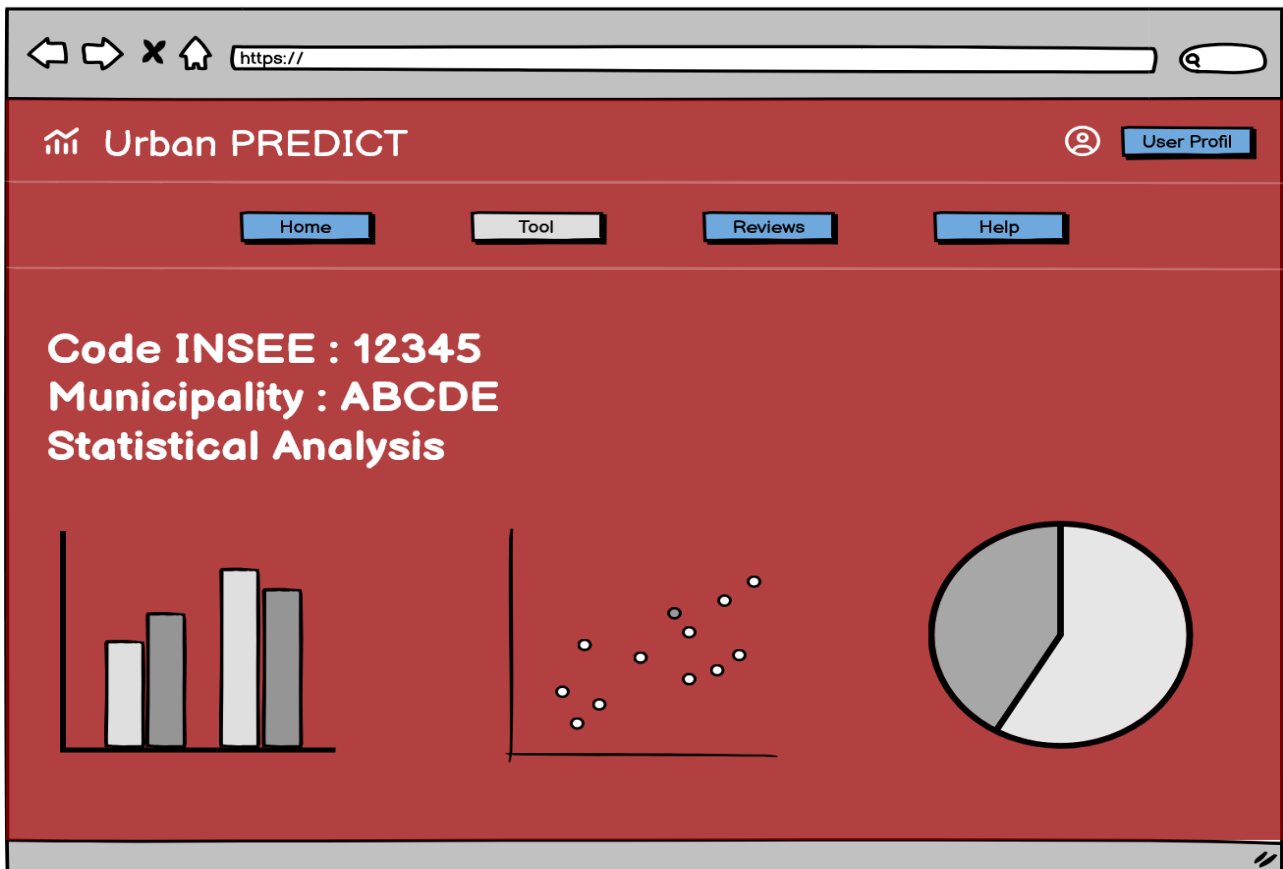
Annexes

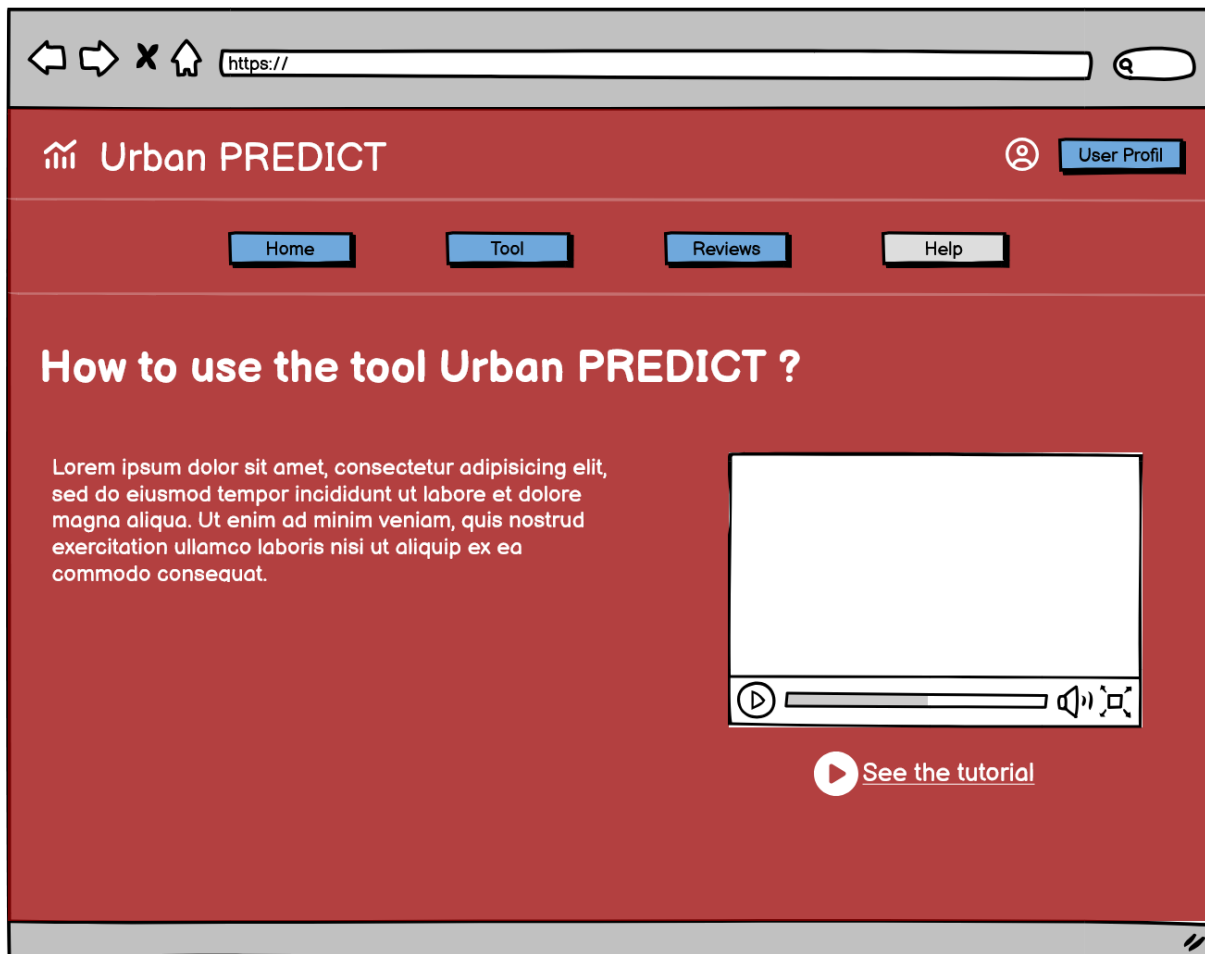
Maquettes de l'interface utilisateur :











Captures d'écran de la version finale de l'interface utilisateur :

Page « Outil » :

Urban PREDICT

Accueil
Outil
Visualisation
À propos
Aide

Outil

Données



1. Sélectionnez une commune.

Entrez un Code INSEE : [Voir les données](#)

Modèle de Machine Learning



2. Utilisez le modèle de Machine Learning.

Entraînez le modèle : [Entraîner](#)

Testez le modèle : [Tester](#)

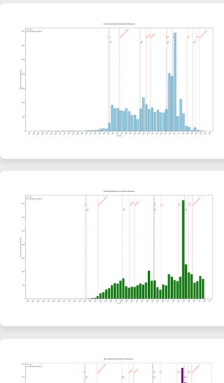
Page « Visualisation » :

Urban PREDICT

Accueil
Outil
Visualisation
À propos
Aide

Visualisation

Sur cette page, vous pouvez visualiser les graphiques et l'analyse exploratoire des données grâce au diaporama interactif.



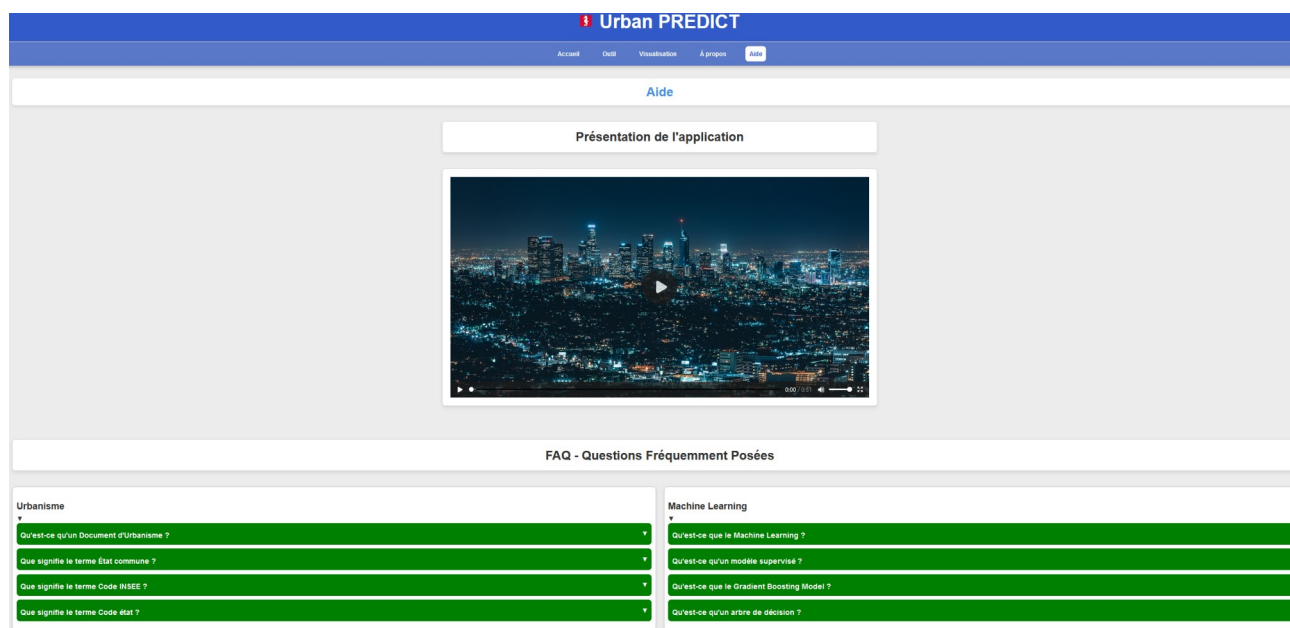
Matrice de Confusion

RNU	729	0	0	0	0	0	0	0	0
RNU / PUI ou PUIJ en élaboration	0	1017	0	0	0	0	0	0	3
RNU / CC en élaboration	0	0	77	0	0	0	0	0	0
PUI ou PUIJ approuvé	0	0	0	2134	0	9	53	0	0
PUI ou PUIJ approuvé - en révision	0	0	0	3	1613	0	159	0	0
CC approuvée	0	0	0	330	0	60	0	0	0
CC approuvée - PUI ou PUIJ en cours	0	0	0	0	76	0	466	0	0

Page « A propos » :



Page « Aide » :



Capture d'écran de la version pour les équipements mobiles :

